

1. Logika cyfrowa

| Układ                       | Formuła / sygnatura                                                 | Opis                                                                                                                              |
|-----------------------------|---------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Sumatory                    |                                                                     |                                                                                                                                   |
| Półsumator                  | $s = x \oplus y$<br>$c = x \& y$                                    | Dodaje dwa bity. <b>s</b> to suma, <b>c</b> to bit przeniesienia (carry).                                                         |
| Pełny sumator (FA)          | $s = x \oplus y \oplus ci$<br>$co = x \& y \mid ci \& (x \oplus y)$ | Jak półsumator, ale przyjmuje też przeniesienie wejściowe <b>ci</b> .                                                             |
| Sumator n-bitowy            | $n \times FA$                                                       | Kaskadowo wyjście $C_i$ trafia na wejście kolejnego FA. Ostatnie <b>co</b> to Carry Out całości.                                  |
| Układy kombinacyjne         |                                                                     |                                                                                                                                   |
| Dekoder                     | $n \rightarrow 2^n$                                                 | Wejście koduje liczbę $k$ . Na wyjściu zapalony jest bit $k$ .                                                                    |
| Multiplekser                | $2^n + n \rightarrow 1$                                             | $2^n$ bitów danych + $n$ bitów sterujących $S$ . Na wyjściu zapalony $S$ -ty bit danych.                                          |
| ALU                         | $A, B, f \rightarrow C$                                             | Dekoder na bitach $f$ wybiera operację np. $A + B$ , multipleksowanie wyników bramkami AND/OR.                                    |
| Układy sekwencyjne (pamięć) |                                                                     |                                                                                                                                   |
| Przerzutnik S-R             | <b>S</b> - set, <b>R</b> - reset                                    | Przechowuje 1 bit ( $Q$ ). <b>S=1</b> ustawia $Q = 1$ , <b>R=1</b> zeruje, <b>S=R=0</b> trzyma stan. Dwa sprzężone <b>NOR</b> -y. |
| Sterowany poziomem          | aktywny gdy <b>CLK</b> = 1                                          | Wejścia <b>S</b> / <b>R</b> zAND-owane z zegarem.                                                                                 |
| Sterowany zboczem           | zbocze 1 $\rightarrow$ 0                                            | Dwa przerzutniki poziomowe w kaskadzie. Stan przepisuje się w momencie zmiany zegara.                                             |

2. Typy danych

| Typ    | char | short | unsigned | int | long | float | double | void* |
|--------|------|-------|----------|-----|------|-------|--------|-------|
| x86-64 | 1    | 2     | 4        | 4   | 8    | 4     | 8      | 8     |

3. Rozszerzanie, obcinanie i przesunięcia

|                    |                                               |
|--------------------|-----------------------------------------------|
| $x \ll k$          | $x \cdot 2^k$ (sgn./uns.)                     |
| $u \gg k$          | $\lfloor u/2^k \rfloor$ - logiczne (zera)     |
| $x \gg k$          | arytmetyczne (kopia MSB), zaokr. do $-\infty$ |
| $x/2^k$ do 0       | $(x + (1 \ll (k-1))) \gg k$                   |
| Strength reduction | $x * 24 \rightarrow (x \ll 5) - (x \ll 3)$    |

4. Operacje i endianness

|                                                                                                                                                                                                                                  |                                             |                       |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------|-----------------------|
| Negacja U2                                                                                                                                                                                                                       | <code>-x = ~x+1 ; -TMin=TMin , -0=0</code>  |                       |
| Wykr. nadmiaru ( <code>s=x+y</code> )                                                                                                                                                                                            | <code>((s^x)&amp;(s^y))&gt;&gt;(w-1)</code> |                       |
| Maska / abs                                                                                                                                                                                                                      | <code>m=x&gt;&gt;31; abs=(x^m)-m</code>     |                       |
| <code>c ? a : b</code>                                                                                                                                                                                                           | <code>b ^ (-c &amp; (a ^ b))</code>         |                       |
| promocja w C: <code>unsigned</code> i <code>int</code> w jednym wyrażeniu/porównaniu<br><code>&lt; &gt; ==</code> ) → <code>int</code> promowany do <code>unsigned</code> (bity bez zmian, <code>-1</code> → <code>Max</code> ). |                                             |                       |
| Schemat                                                                                                                                                                                                                          | Najniższy adres                             | <code>0x0123</code>   |
| Big Endian                                                                                                                                                                                                                       | MSB                                         | <code>[01][23]</code> |
| Little Endian                                                                                                                                                                                                                    | LSB                                         | <code>[23][01]</code> |

5. Zmiennoprzecinkowe (IEEE 754)

$V = (-1)^s \times M \times 2^E$       Bias =  $2^{k-1} - 1$

| Format | bity | s | exp ( $k$ ) | frac ( $n$ ) |
|--------|------|---|-------------|--------------|
| float  | 32   | 1 | 8           | 23           |
| double | 64   | 1 | 11          | 52           |

| Kategoria      | exp                              | Własności                                                                                                                                                    |
|----------------|----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Znormalizowane | $!= 0 \dots 0$<br>$!= 1 \dots 1$ | $E = \text{exp} - \text{Bias}$ . $M = 1.\text{frac}$ .<br>Najgęściej ułożone wokół 0.                                                                        |
| Zdenormaliz.   | $== 0 \dots 0$                   | $E = 1 - \text{Bias}$ . $M = 0.\text{frac}$ .<br>Reprezentują $\pm 0.0$ (frac = 0) i liczby bardzo bliskie zera.                                             |
| Specjalne      | $== 1 \dots 1$                   | $\text{frac} == 0 \rightarrow \pm \infty$ (nadmiar, dzielenie przez 0).<br>$\text{frac} != 0 \rightarrow \text{NaN}$ (np. $\sqrt{-1}$ , $\infty - \infty$ ). |

Zaokrąglenie GRS i Round-to-Even  
Domyślny tryb to **Round-to-Even** (zapobiega błędowi statycznemu przy sumowaniu).

|                                                                                                                                                           |         |                                                                   |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------|---------|-------------------------------------------------------------------|
| <div>... x x <b>G</b>    <b>!</b>    <b>R S S S</b> ...</div> <div><b>G</b> - ostatni zachowany, <b>R</b> - pierwszy usunięty, <b>S</b> - 0R reszty</div> |         |                                                                   |
| Warunek                                                                                                                                                   | Ułamek  | Akcja                                                             |
| <b>R=0</b>                                                                                                                                                | $< 0.5$ | W dół (odcięcie)                                                  |
| <b>R=1, S=1</b>                                                                                                                                           | $> 0.5$ | W górę (+1 do <b>G</b> )                                          |
| <b>R=1, S=0</b>                                                                                                                                           | $= 0.5$ | <b>Do parzystej:</b> w górę gdy <b>G=1</b> , w dół gdy <b>G=0</b> |

|                                              |                                                                                    |
|----------------------------------------------|------------------------------------------------------------------------------------|
| Własności matematyczne i rzutowanie          |                                                                                    |
| Brak łączności                               | $(a + b) + c \neq a + (b + c)$ - zaokrąglenia                                      |
| Brak rozdzielności                           | $a(b + c) \neq ab + ac$                                                            |
| <b>int</b> $\rightarrow$ <b>float</b>        | możliwa utrata precyzji (zaokrągła)                                                |
| <b>int</b> $\rightarrow$ <b>double</b>       | bezstratne ( $52 > 32$ bity)                                                       |
| <b>float/double</b> $\rightarrow$ <b>int</b> | obcina do 0; poza zakresem lub <b>NaN</b> $\rightarrow$ <b>TMin</b> ( 0x80000000 ) |

## 6. Rejestry x86\_64

|                                                            |                                                            |
|------------------------------------------------------------|------------------------------------------------------------|
| <div>%rax</div> <div>%eax</div> <div>%ax<br/>%ah %al</div> | <div>%rbx</div> <div>%ebx</div> <div>%bx<br/>%bh %bl</div> |
| <div>%rcx</div> <div>%ecx</div> <div>%cx<br/>%ch %cl</div> | <div>%rdx</div> <div>%edx</div> <div>%dx<br/>%dh %dl</div> |
| <div>%rsi</div> <div>%esi</div> <div>%si<br/>%sil</div>    | <div>%rdi</div> <div>%edi</div> <div>%di<br/>%dil</div>    |
| <div>%rbp</div> <div>%ebp</div> <div>%bp<br/>%bpl</div>    | <div>%rsp</div> <div>%esp</div> <div>%sp<br/>%spl</div>    |
| <div>%r8</div> <div>%r8d</div> <div>%r8w<br/>%r8b</div>    | <div>%r9</div> <div>%r9d</div> <div>%r9w<br/>%r9b</div>    |

**Uwaga:** Rejestry od %r10 do %r15 używają schematu:  
%r10, %r10d, %r10w, %r10b.

### Podział ról rejestrów

|                                                                             |                                                                                                                                                     |
|-----------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| <div>%rax</div> <div>%rdi,<br/>%rsi,<br/>%rdx,<br/>%rcx,<br/>%r8, %r9</div> | Akumulator. Główny rejestr arytmetyczny. Przechowuje <b>wartość zwracaną</b> .<br><br>Kolejno: <b>od 1. do 6. argumentu funkcji</b> . Caller-saved. |
| <div>%rsp</div>                                                             | Wskaźnik stosu (SP). Wskazuje wierzchołek ramki.                                                                                                    |
| <div>%rbp</div>                                                             | Wskaźnik bazy (BP). Początek ramki stosu. Callee-saved.                                                                                             |
| <div>%rbx,<br/>%r12-%r15</div>                                              | Ogólnego przeznaczenia. Wszystkie <b>callee-saved</b> .                                                                                             |
| <div>%rip</div>                                                             | Wskaźnik instrukcji (Program Counter).                                                                                                              |

## 7. Tryby adresowania (operandy)

| Tryb                   | Format                    | Obliczanie adresu         |
|------------------------|---------------------------|---------------------------|
| Natychmiastowy (Imm)   | <div>\$Imm</div>          | <div>Imm</div>            |
| Rejestrowy (Reg)       | <div>%Ra</div>            | <div>%Ra</div>            |
| Bezpośredni (Mem)      | <div>Imm</div>            | <div>M[Imm]</div>         |
| Pośredni (Mem)         | <div>(%Rb)</div>          | <div>M[%Rb]</div>         |
| Z przesunięciem        | <div>D(%Rb)</div>         | <div>M[%Rb + D]</div>     |
| Skalowany (Ind/scaled) | <div>D(%Rb, %Ri, S)</div> | <div>M[%Rb+%Ri*S+D]</div> |
| Skalowany (baseless)   | <div>(, %Ri, S)</div>     | <div>M[%Ri * S]</div>     |

**Legenda:** Imm / D to stała liczbowa, %Ra / %Rb to rejestr bazowy, %Ri to rejestr indeksowy, a S to skala (tylko: 1, 2, 4 lub 8).

## 8. Katalog instrukcji (AT&T)

| Opcode                    | Efekt                         | Opis                                                    |
|---------------------------|-------------------------------|---------------------------------------------------------|
| <div>mov S, D</div>       | $D \leftarrow S$              | Kopiuje wartość z S do D.                               |
| <div>lea S, D</div>       | $D \leftarrow \text{addr}(S)$ | Oblicza adres S (bez czytania pamięci).                 |
| <div>add S, D</div>       | $D \leftarrow D + S$          | Dodawanie (ustawia flagi).                              |
| <div>sub S, D</div>       | $D \leftarrow D - S$          | Odejmowanie (ustawia flagi).                            |
| <div>imul S, D</div>      | $D \leftarrow D * S$          | Mnożenie liczb ze znakiem.                              |
| <div>sal / shl k, D</div> | $D \ll= k$                    | Przesunięcie w lewo (mnożenie przez $2^k$ ).            |
| <div>sar k, D</div>       | $D \gg= k$                    | Arytmetyczne w prawo (dzielenie). <b>Kopiuje znak</b> . |
| <div>shr k, D</div>       | $D \gg= k$                    | Logiczne w prawo. Dopełnia zerami.                      |
| <div>and / or S, D</div>  | $D \leftarrow D \& /   S$     | Operacje bitowe (ustawiają flagi).                      |
| <div>xor S, D</div>       | $D \leftarrow D \wedge S$     | Często jako <b>xor %rax, %rax</b> do zerowania.         |

**Uwaga o sufiksach:** Instrukcje przyjmują sufiks określający rozmiar danych: **b** (8-bit), **w** (16-bit), **l** (32-bit), **q** (64-bit).  
Np. `movl` kopiuje 32 bity (i automatycznie zeruje górną połowę 64-bitowego rejestru!).